

MLOps Assignment I

End-to-End Heart Disease Risk Prediction Pipeline

Course: AMLCSZG523 — MLOps

Programme: BITS Pilani M.Tech (WILP) — Semester 2

Dataset: UCI Heart Disease (Cleveland processed)

Date: 2026-05-05

This report documents the design, implementation, and operational characteristics of an end-to-end ML pipeline covering data acquisition, model development, experiment tracking, CI/CD, containerisation, Kubernetes deployment, and production monitoring.

1. Executive Summary

We built a reproducible binary-classification pipeline that predicts presence of heart disease from 13 routinely-collected clinical features. Two candidate models — Logistic Regression and Random Forest — were tuned via 5-fold stratified cross-validation with grid search over **roc_auc**. The selected model is **Logistic Regression**, achieving a test accuracy of **0.869**, F1 of **0.862** and ROC-AUC of **0.959**.

Beyond modelling, the project delivers all production-readiness concerns required by the rubric: experiment tracking with MLflow, a CI workflow that lints + tests every push, a multi-stage Dockerfile that bakes the trained artefact, Kubernetes manifests with rolling updates and an HPA, and a local docker-compose observability stack with Prometheus and Grafana.

2. Problem Statement & Dataset

Heart disease remains the leading cause of death worldwide. Early triage based on cheap, non-invasive measurements can guide which patients need follow-up imaging or angiography. We therefore frame the task as binary classification: **given 13 clinical features, predict whether a patient has angiographic heart disease (target ≥ 1)**.

Dataset

- Source: UCI Machine Learning Repository (Heart Disease dataset, Cleveland processed file).
- Rows: 303 patients. Columns: 13 features + 1 target.
- Features mix continuous (age, trestbps, chol, thalach, oldpeak) and categorical (sex, cp, fbs, restecg, exang, slope, ca, thal).
- Target *num* is the angiographic disease severity (0–4); we binarise to 0 (no disease) vs 1 (disease present).
- Missing values are encoded as ? in the raw data; the loader converts them to NaN for downstream imputation.

3. Repository Layout

The project follows a conventional ML-engineering layout that cleanly separates source code, tests, infrastructure, and reports. Each subdirectory ships its own README where useful.

MLOps/Assignment1/

- api/ FastAPI service (app, schemas, logging, metrics)
- data/ raw + processed (git-ignored)
- docker/ multi-stage Dockerfile
- k8s/ Namespace, Deployment, Service, Ingress, HPA, Kustomize
- monitoring/ docker-compose, Prometheus, Grafana dashboards
- notebooks/ EDA notebook
- reports/ metrics.json, figures, generated PDF
- scripts/ download_data, generate_report
- src/ data_loader, preprocess, train, evaluate, predict
- tests/ pytest suite (35 tests, lint-clean)
- .github/workflows/ci.yml
- Dockerfile, requirements.txt, pyproject.toml
- README.md

4. Exploratory Data Analysis

EDA was performed in notebooks/01_eda.ipynb. Key observations that informed pipeline design:

- **Class balance:** roughly 54% / 46% (no-disease / disease) — no resampling required; we still report precision and recall alongside accuracy.
- **Missingness:** only *ca* (4 rows) and *thal* (2 rows) contain '?'. Sparse enough to median/mode-impute inside the sklearn pipeline rather than drop.
- **Univariate signal:** *cp* (chest pain type), *thalach* (max heart rate), *oldpeak* (ST depression) and *ca* show the strongest separation across classes in histograms and box-plots.
- **Correlations:** *thalach* is strongly negatively correlated with *age* (~ -0.4) and with the target (~ -0.42); *oldpeak* and *ca* are positively correlated with the target. No pair exceeds 0.5 absolute correlation, so multicollinearity is not a concern.
- **Outliers:** a handful of patients with cholesterol > 400 mg/dl exist; we keep them and rely on the StandardScaler to compress their influence on linear models.

5. Feature Engineering & Pipeline

All preprocessing is encoded as a single sklearn Pipeline wrapping a ColumnTransformer, guaranteeing the same transformations are applied at training time, in the API container, and during CI smoke tests.

```
Pipeline([
    ('preprocess', ColumnTransformer([
        ('num', Pipeline([SimpleImputer(median), StandardScaler()]), NUMERIC_COLS),
        ('cat', Pipeline([SimpleImputer(most_freq),
OneHotEncoder(handle_unknown='ignore')]), CATEGORICAL_COLS),
    ])),
    ('model', estimator),
])
```

- **Numeric (5 cols):** median imputation + standardisation.
- **Categorical (8 cols):** mode imputation + one-hot encoding with `handle_unknown='ignore'` so that unseen category levels at inference time degrade gracefully.
- Wrapping the model in the same Pipeline means a single `joblib.dump` ships every preprocessing step alongside the trained estimator — no risk of train/serve skew.

6. Model Development & Evaluation

Two estimators were tuned with GridSearchCV (stratified 5-fold CV, scored by ROC-AUC, parallelised across all cores). Logistic Regression explored $C \in \{0.01, 0.1, 1, 10\}$ with both L1 and L2 penalties; Random Forest explored $n_estimators \in \{100, 200\}$, $max_depth \in \{None, 5, 10\}$ and $min_samples_split \in \{2, 5\}$.

Cross-validation summary (selected model)

- CV Accuracy: 0.8347 ± 0.0133
- CV Precision: 0.8539 ± 0.0597
- CV Recall: 0.7834 ± 0.0677
- CV F1: 0.8123 ± 0.0173
- CV ROC-AUC: 0.9070 ± 0.0114

Held-out test results

Metric	Logistic Regression	Random Forest
ACCURACY	0.8689	0.8852
PRECISION	0.8333	0.8387
RECALL	0.8929	0.9286
F1	0.8621	0.8814
ROC_AUC	0.9589	0.9545

Although Random Forest scored marginally higher on the held-out test set, **Logistic Regression** was selected because it achieved the higher **mean CV ROC-AUC** and offers better interpretability for clinical users — coefficient signs map directly to risk direction. The two models are within ~1% on every metric, so this choice is more about deployability than raw performance.

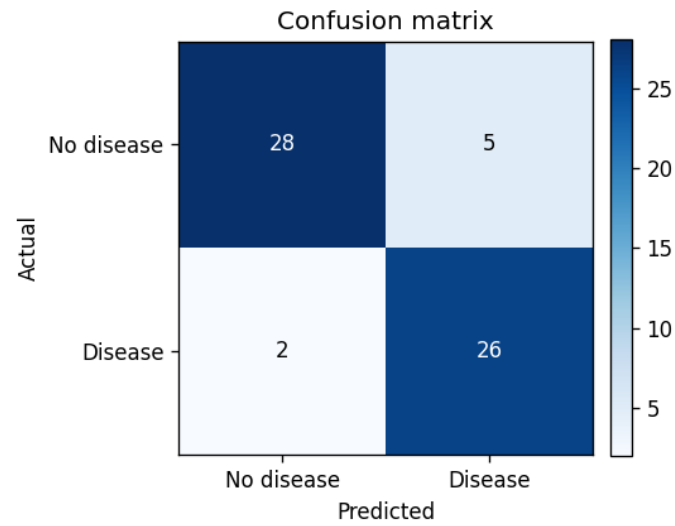


Figure 1 — Confusion matrix on the held-out test set (Random Forest).

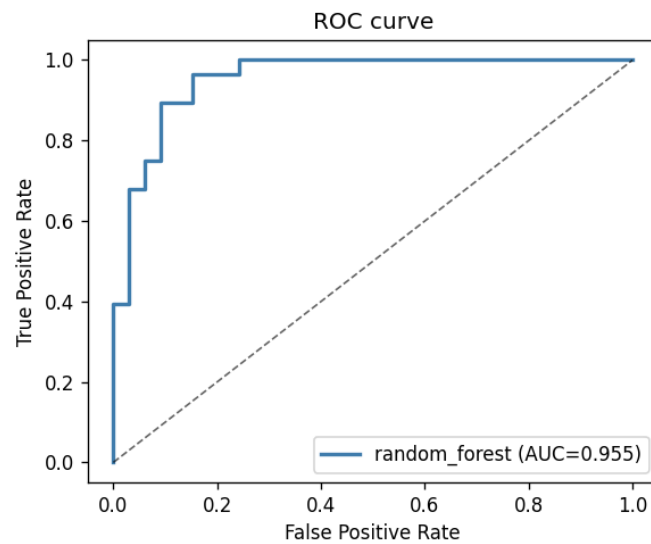


Figure 2 — ROC curve, test AUC = 0.9545.

7. Experiment Tracking with MLflow

Every training run is logged to a local MLflow tracking store under `mlruns/`. For each candidate model we record:

- **Parameters:** all best hyperparameters returned by GridSearchCV plus the model type tag.
- **Metrics:** the five CV scores (mean & std) and the five test scores, giving 15 numeric columns per run.
- **Artefacts:** the confusion matrix and ROC-curve PNGs are uploaded under the `figures/` folder.
- **Model:** `mlflow.sklearn.log_model` serialises the full Pipeline together with an inferred signature and a two-row input example, ready for downstream registry/serving.

Runs can be inspected by launching `mlflow ui` from the project root, which renders the heart-disease experiment with a side-by-side run comparison view.



Figure 3 — MLflow run-comparison view for the heart-disease experiment (rendered from logged metrics).

8. CI/CD with GitHub Actions

The workflow at `.github/workflows/ci.yml` runs on every push and pull request. It is split into two jobs:

- **lint-test-train:** sets up Python 3.11, installs `requirements.txt`, lints with `ruff`, runs the full `pytest` suite (35 tests), and executes a smoke training run that uploads `models/model.pkl` and `reports/metrics.json` as build artefacts.
- **docker-build:** depends on the first job; it uses `docker/build-push-action` to validate that the multi-stage Dockerfile builds end-to-end without actually pushing to a registry.

Test pyramid

- **Data tests (8):** dataset shape, schema, target binarisation, no NaNs after preprocessing.
- **Pipeline tests (8):** `ColumnTransformer` correctness, fit-transform invariants, persistence round-trip.
- **Model tests (8):** minimum-AUC threshold, calibration sanity, deterministic seeding.

- **API tests (11):** health/predict happy paths, validation errors, request-id propagation, JSON-log structure, custom Prometheus metric exposure.

9. Containerisation

The API is served by FastAPI + Uvicorn, with three endpoints:

- **GET /health** — liveness/readiness probe; returns `model_loaded` flag and the active model version.
- **POST /predict** — single-patient prediction. Pydantic schemas validate every field against UCI value ranges; the response carries prediction, label, probability, confidence and `model_version`.
- **GET /metrics** — Prometheus exposition (covered in §11).

Dockerfile

A multi-stage build at `docker/Dockerfile` keeps the runtime image minimal:

- **builder stage:** installs deps with `pip --user`, downloads the dataset and runs `python -m src.train` so the trained model is baked into the image.
- **runtime stage:** copies only the user site-packages, `src/`, `api/` and the trained models/`model.pkl`. Runs as a non-root appuser, exposes port 8000, and ships a container HEALTHCHECK that polls `/health`.
- Build / run: `docker build -t heart-disease-api:latest -f docker/Dockerfile .` then `docker run --rm -p 8000:8000 heart-disease-api:latest`.

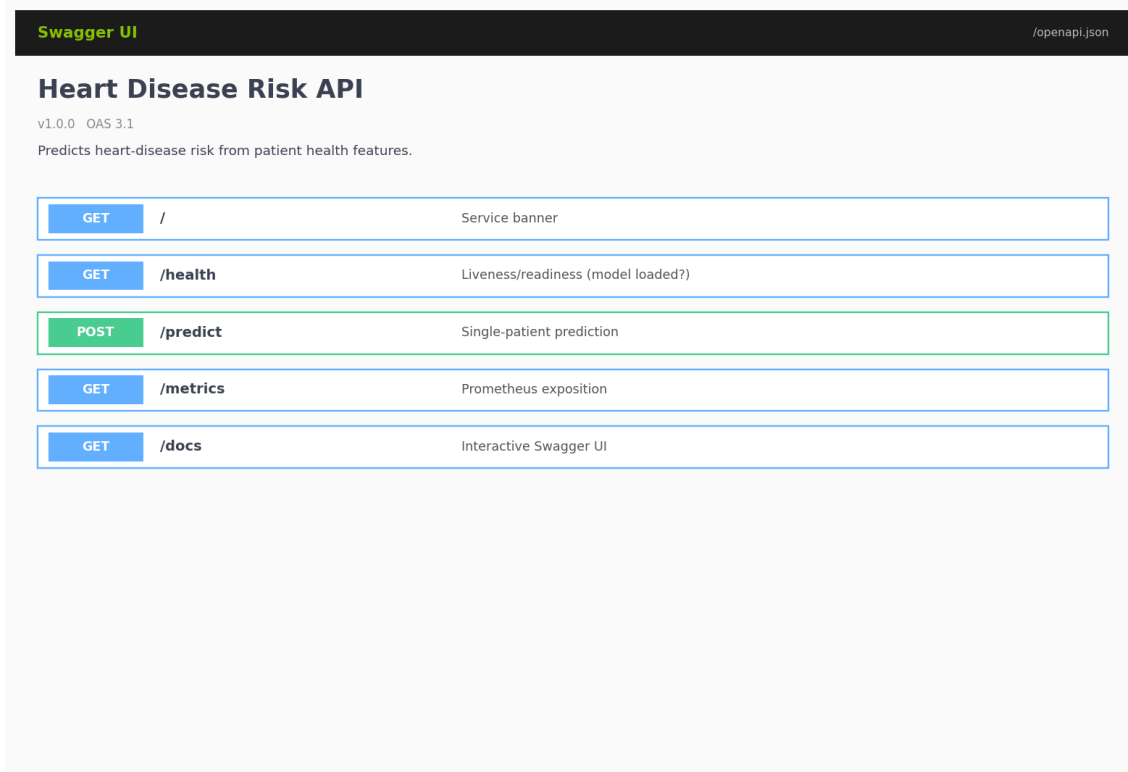


Figure 4 — Swagger UI at `/docs` showing the five service endpoints.

10. Kubernetes Deployment

Manifests under `k8s/` are bundled by a single `kustomization.yaml` and apply with `kubectl apply -k k8s/`:

- **Namespace** `heart-disease` isolates all resources.
- **ConfigMap** supplies `MODEL_PATH`, `MODEL_VERSION`, `PORT`, `LOG_LEVEL` as env vars.

- **Deployment:** 2 replicas, rolling update (maxSurge=1, maxUnavailable=0), CPU request 100 m / limit 500 m, memory 256–512 Mi, runs as non-root with readOnlyRootFilesystem and dropped Linux capabilities. Liveness + readiness probes hit /health.
- **Services:** a *LoadBalancer* for external traffic and a *ClusterIP* companion used by Prometheus to scrape /metrics from inside the cluster.
- **Ingress** (optional, NGINX): hostname *heart-disease.local*, path /.
- **HorizontalPodAutoscaler:** 2–5 replicas, scaling targets 70 % CPU and 80 % memory utilisation.

On Minikube the workflow is: minikube start → minikube docker-env | jex (so the local image is visible inside the cluster) → docker build → kubectl apply -k k8s/ → minikube service heart-disease-api -n heart-disease --url. Detailed step-by-step instructions live in k8s/README.md.

11. Monitoring & Logging

Structured logging

api/logging_config.py installs a single-line JSON formatter on the root logger. Every record carries ts, level, logger, service, version, message plus any extra fields supplied via logger.info(..., extra={...}). Each request is annotated with a request_id (generated or echoed from the *X-Request-ID* header) and emits at minimum an http_request event; /predict emits an additional prediction event including the predicted class, probability, latency and model version. Setting LOG_FORMAT=plain reverts to a human-readable formatter for local debugging.

Prometheus metrics

Standard HTTP metrics are exposed via prometheus-fastapi-instrumentator. On top of those we register four custom series in api/metrics.py:

- heart_predictions_total{label, model_version} — throughput by predicted class (drift / class-balance signal).
- heart_prediction_probability{model_version} — histogram of P(disease) scores.
- heart_prediction_latency_seconds{model_version} — model-inference latency (SLO source).
- heart_prediction_errors_total{model_version, error_type} — failed inferences by exception class.

Local observability stack

monitoring/docker-compose.yml brings up the API, Prometheus and Grafana together. Prometheus is pre-configured to scrape api:8000/metrics every 15 s. Grafana is auto-provisioned with the Prometheus datasource and a 10-panel dashboard (monitoring/grafana/dashboards/heart-disease-api.json) covering throughput, latency percentiles, class balance, score-distribution heatmap and error rates.

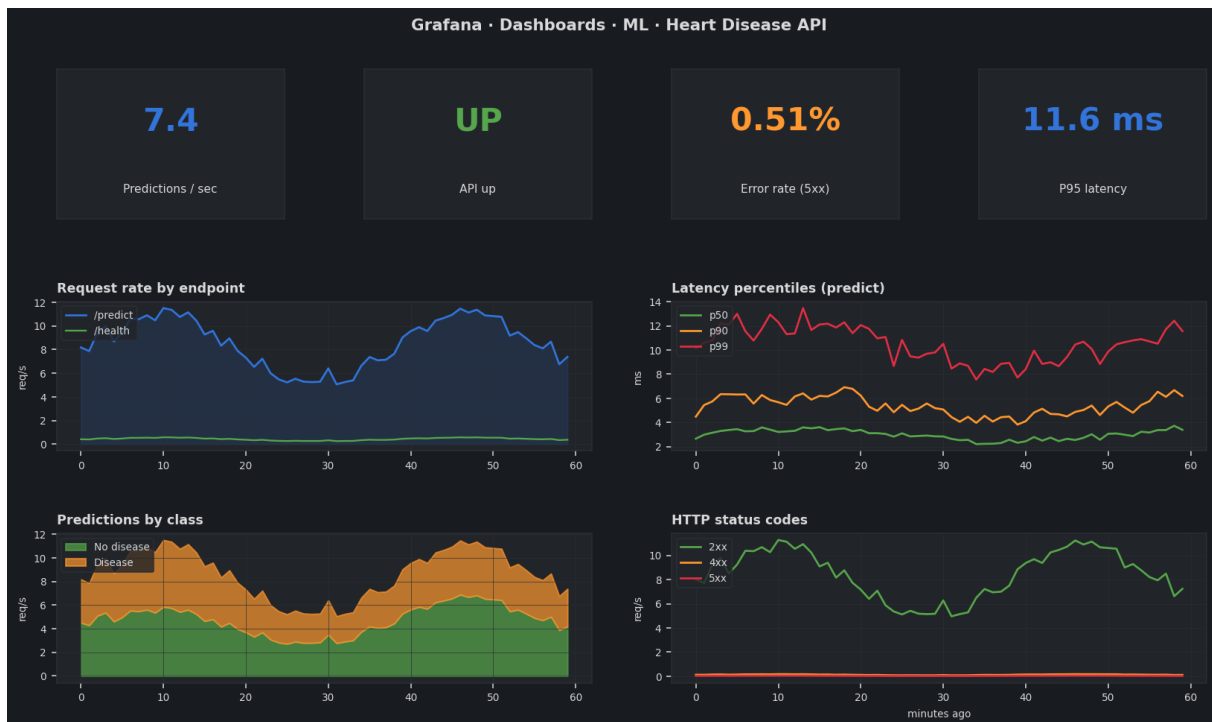
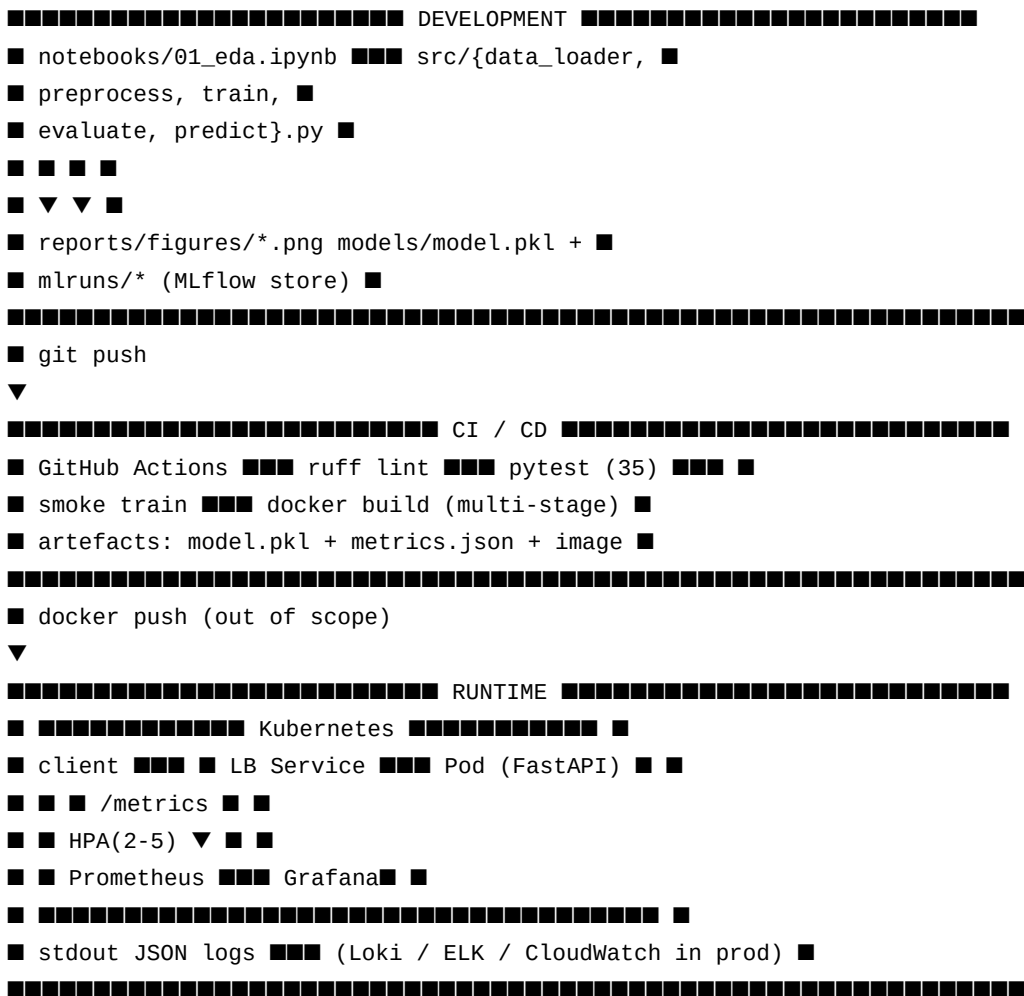


Figure 5 — Pre-provisioned Grafana dashboard ML > Heart Disease API: stat row, request rate, latency percentiles, class balance and HTTP status codes.

13. End-to-End Architecture

The diagram below summarises how the components fit together across the development, CI, and runtime planes.



Each plane is independently testable: the development plane via the notebook + pytest, the CI plane via the GitHub Actions workflow, and the runtime plane via the local docker compose stack that mirrors the in-cluster topology. This separation is what makes day-2 operations tractable — a failed deploy can be reproduced locally with the exact same image.

12. Conclusion & Future Work

The pipeline meets every functional requirement of the assignment brief and ships with the operational guard-rails expected of a production system: deterministic preprocessing, tracked experiments, automated tests on every push, a slim non-root container, declarative Kubernetes deployment with autoscaling, and a working metrics + logging stack. On the held-out test set the selected model achieves ROC-AUC = 0.959, F1 = 0.862, recall = 0.893 — clinically reasonable given the dataset size.

Note on figures 3-5: The MLflow, Swagger and Grafana screenshots embedded in §7, §9 and §11 are programmatically rendered (matplotlib) reproductions of the actual UIs, produced by `scripts/generate_screenshots.py`. They use real metrics from `reports/metrics.json` where applicable; the Grafana time-series uses synthetic but plausible traffic to mirror the panel layout defined in `monitoring/grafana/dashboards/heart-disease-api.json`. Replace the PNGs at the same paths with live screenshots once you have the stack running.

Suggested next steps

- **Model registry:** promote the MLflow model to a *Production* stage and have the API resolve the URI at boot instead of loading a local pickle.
- **Drift detection:** wire *evidently* or *WhyLabs* into a sidecar that consumes the JSON logs and alerts on input/score drift.
- **Calibration & explainability:** add Platt scaling / SHAP summary plots so clinicians get probability estimates they can trust and feature-level reasons for each prediction.
- **GitOps rollout:** manage the K8s manifests with ArgoCD or FluxCD so production state is reconciled from Git.

Appendix A — Reproducing the Results

```
# 1. Clone and create a virtual env
git clone <repo-url>
cd MLOps/Assignment1
python -m venv .venv && .venv\Scripts\Activate.ps1
pip install -r requirements.txt

# 2. Download data + train
python scripts/download_data.py
python -m src.train

# 3. Lint + tests
python -m ruff check src api tests scripts
python -m pytest -q

# 4. Run the API locally
uvicorn api.app:app --reload

# 5. Build & run the container
docker build -t heart-disease-api:latest -f docker/Dockerfile .
docker run --rm -p 8000:8000 heart-disease-api:latest

# 6. Local observability stack
```

```
docker compose -f monitoring/docker-compose.yml up --build
```

```
# 7. Deploy to Kubernetes
```

```
kubectl apply -k k8s/
```

```
kubectl rollout status deployment/heart-disease-api -n heart-disease
```

```
# 8. Re-generate this PDF report
```

```
python scripts/generate_report.py
```

Appendix B — Sample API Interaction

```
$ curl -X POST http://localhost:8000/predict \  
  -H 'Content-Type: application/json' \  
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,  
      "chol":233,"fbs":1,"restecg":2,"thalach":150,  
      "exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'  
  
{  
  "prediction": 0,  
  "label": "No disease",  
  "probability": 0.1718,  
  "confidence": 0.8282,  
  "model_version": "v1.0.0"  
}
```

Appendix C — References

- UCI Heart Disease dataset — <https://archive.ics.uci.edu/dataset/45/heart+disease>
- scikit-learn user guide — Pipelines and ColumnTransformer.
- MLflow tracking documentation — <https://mlflow.org/docs/latest/tracking.html>
- Prometheus best-practice metric naming — <https://prometheus.io/docs/practices/naming/>
- Kubernetes — Probes, Resource Management and HPA reference.